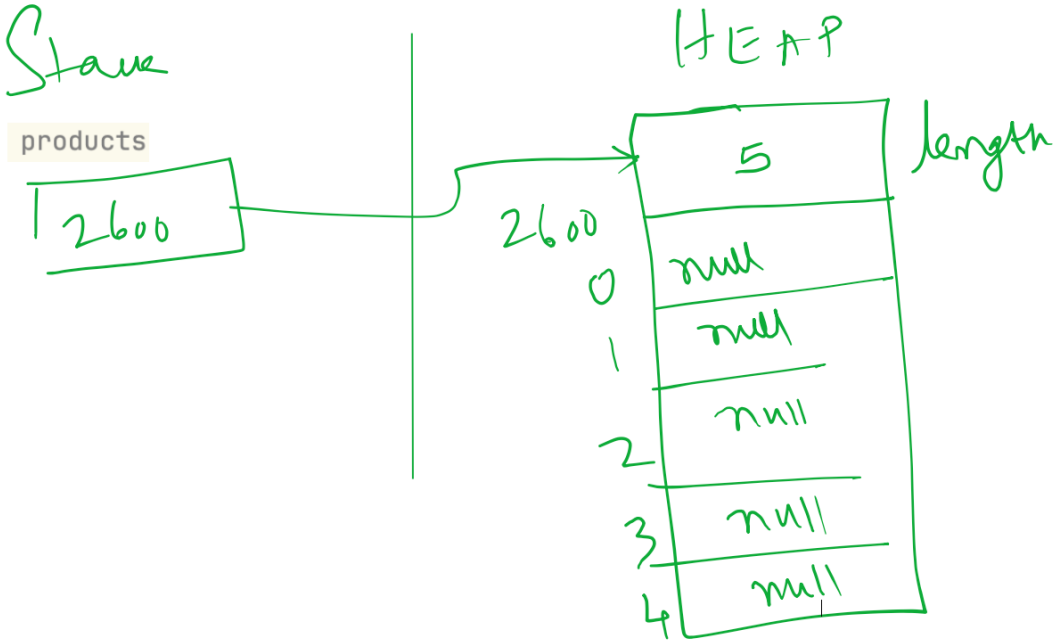
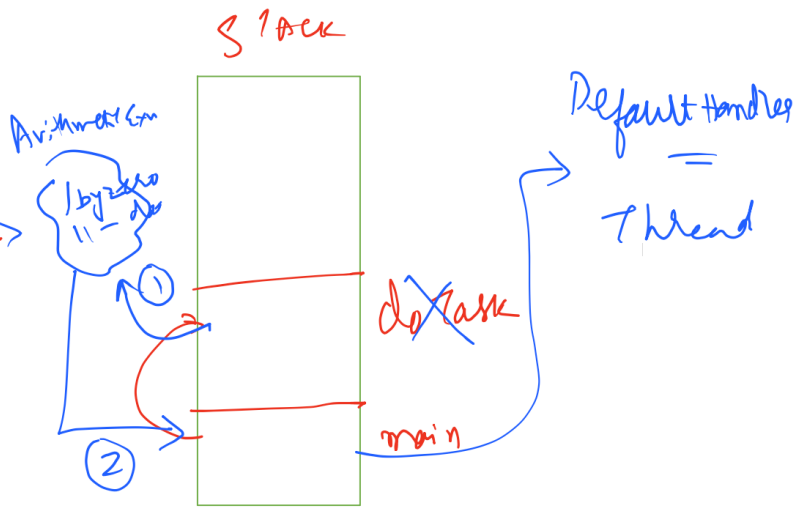


- getId(): int
- setId(int): void
- getName(): String
- setName(String): void
- getPrice(): double
- setPrice(double): void
- isExpensive(): boolean
- toString(): String ↑Object
- getConnectivity(): String
- setConnectivity(String): void
- isExpensive(): boolean ↑Product
- toString(): String ↑Product
- getClass(): Class<?>
- hashCode(): int
- equals(Object): boolean
- clone(): Object
- toString(): String
- notify(): void
- notifyAll(): void
- wait(): void
- wait(long): void
- wait0(long): void
- wait(long, int): void

- getId(): int
- setId(int): void
- getName(): String
- setName(String): void
- getPrice(): double
- setPrice(double): void
- isExpensive(): boolean
- toString(): String ↑Object
- isExpensive(): boolean ↑Product
- getScreenType(): String
- setScreenType(String): void
- getClass(): Class<?>
- hashCode(): int
- equals(Object): boolean
- clone(): Object
- toString(): String
- notify(): void
- notifyAll(): void
- wait(): void
- wait(long): void
- wait0(long): void
- wait(long, int): void

```java
public class FirstExample {
    public static void main(String[] args) {
        System.out.println("Start!!!");
            doTask();
        System.out.println("End");
    }

    private static void doTask() {
        int x = 10;
        int y = 0;
        int res = x / y;
        System.out.println("Result " + res);
    }
}
```



Scenario 1:

```
client

User user = new User(..);
//
userDao.register(user);
```

```java
interface UserDao {
    void register(User user);
}
```

```java
class UserDaoMongoImpl implements UserDao {
    void regiser(User user) {
        try {
            ...
        } catch(MongoException ex) {
            // handle
        }
    }
}
```

Scenario 2:

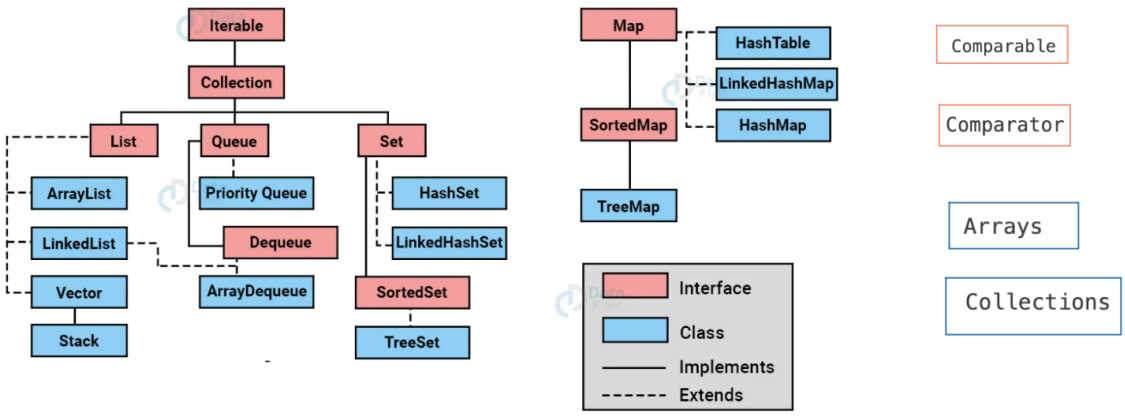
```
client

User user = new User(..);
//
try {
userDao.register(user);
} catch(MongoException ex) {
    ex.getMessage(); // ex.printStackTrace();
}
```

```java
interface UserDao {
    void register(User user) throws MongoException;
}
```

```java
class UserDaoMongoImpl implements UserDao {
    void regiser(User user) throws MongoException {

    }
}
```



Scenario	ArrayList	LinkedList
Random access heavy	✅ O(1)	❌ O(n)
Frequent insert at beginning	❌ O(n)	✅ O(1)
Memory efficiency	✅ Better	❌ Extra node objects
CPU cache friendly	✅ Yes	❌ No
Iteration speed	✅ Faster	❌ Slower